Loan Default Risk Prediction

Complete project documentation · generated 2026-10-14 · project root: `/home/sachin/Desktop/loan-default-risk`

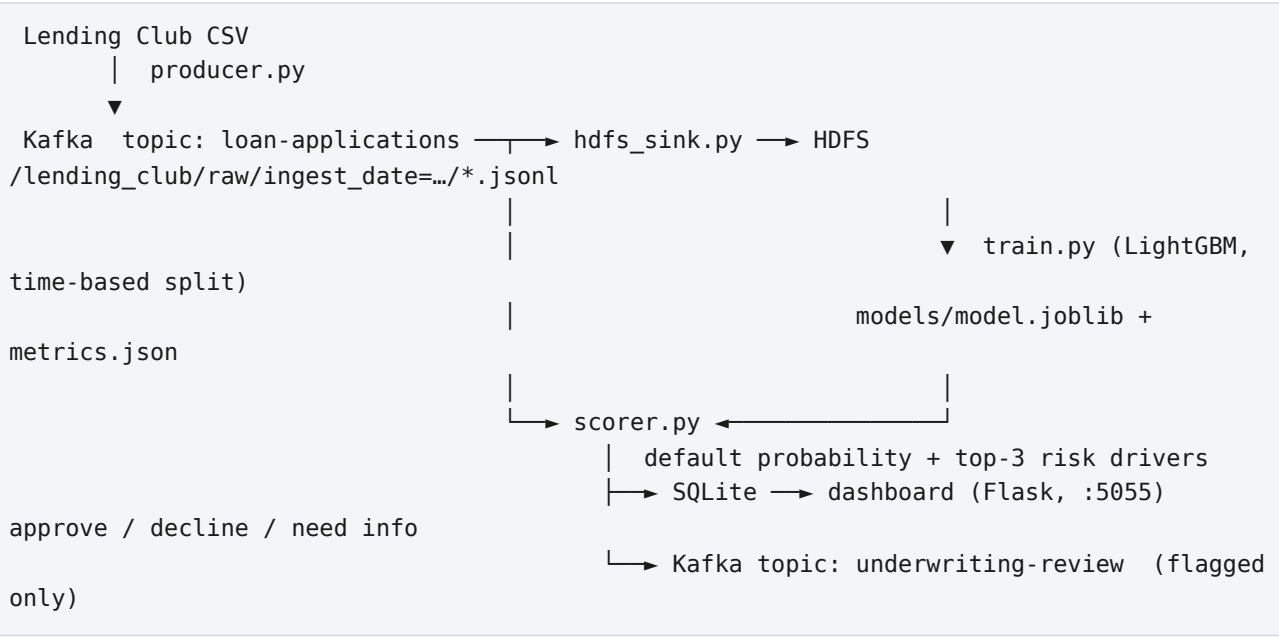
1. Overview

This project replaces a fixed credit-score cutoff with a machine-learning model that scores each loan application on the **combined** effect of income, debt ratio, credit history and loan terms. Applications whose predicted default probability is **greater than 0.5** are flagged for manual underwriting in a web dashboard, where an underwriter can approve, decline, or request more information.

It is an end-to-end streaming pipeline built on the Lending Club loan dataset: Kafka for ingestion, HDFS for the raw data lake, LightGBM for modelling, SQLite for scored results, and a Flask dashboard for the review queue.

Item	Detail
Language / runtime	Python 3 (bytecode present for CPython 3.12), virtualenv at `.venv/`
Package	`src/lrisk` (run as `python -m lrisk.<module>` with `PYTHONPATH=src`)
Model	LightGBM gradient-boosted trees (binary classifier) with SHAP-style reason codes
Infrastructure	Docker Compose: single-node Kafka 3.9.0 (KRaft) + single-node HDFS (Hadoop 3.3.6)
Dashboard	Flask on `127.0.0.1:5055`
Source size	~1,070 lines across 12 Python modules, 1 HTML template, 1 test file
Version control	Not a git repository

2. Architecture



Data flow

- **History path:** the producer streams loans issued before the split date (default `2017-01`) with labels into Kafka; the HDFS sink persists them as JSON-lines; the trainer reads them from HDFS and produces the model bundle and metrics.
- **Live path:** the producer streams 2017+ loans with the `loan_status` label stripped (`--live`); the scorer consumes them, computes the default probability and top-3 risk drivers, upserts into SQLite and publishes flagged items to `underwriting-review`.
- **Review path:** the dashboard reads SQLite and lets underwriters record decisions on flagged applications only.

3. Repository layout

Path	Purpose
`README.md`	Quick start, design decisions, threshold discussion
`Makefile`	Task runner: up, down, reset, sample, load, train, live, dashboard, test, demo
`docker-compose.yml`, `hadoop.env`	Kafka (KRaft) + HDFS NameNode/DataNode definitions and Hadoop config
`requirements.txt`	Python dependencies
`scripts/download_dataset.sh`	Downloads the real dataset from Kaggle (`wordsforthewise/lending-club`)
`src/lrisk/config.py`	Central configuration via environment variables
`src/lrisk/producer.py`	CSV → Kafka

`src/lrisk/hdfs_sink.py`, `hdfs_client.py`	Kafka → HDFS micro-batches; minimal WebHDFS client
`src/lrisk/features.py`	Cleaning, target creation, feature engineering (shared by train and score)
`src/lrisk/train.py`	HDFS → features → LightGBM → `models/`
`src/lrisk/model.py`	`ModelBundle`: booster + category vocabulary + threshold; scoring and reason codes
`src/lrisk/scorer.py`	Kafka → score → SQLite + review topic
`src/lrisk/store.py`	SQLite schema and data access
`src/lrisk/kafka_utils.py`	Topic creation (3 partitions, replication factor 1)
`src/lrisk/dashboard/`	Flask app (`app.py`) and Jinja template (`templates/index.html`)
`src/lrisk/generate_sample.py`	Synthetic Lending Club-format data generator (demos/tests only)
`tests/test_pipeline.py`	6 offline tests (no Kafka/HDFS needed)
`models/`	`model.joblib` (~477 KB), `metrics.json`
`data/`	`sample_lending_club.csv` (60,000 synthetic rows, 10 MB), `accepted_2007_to_2018Q4.csv.gz` (real dataset, ~393 MB, 2,260,702 lines uncompressed), `scored.db`

4. Setup and usage

Quick start (synthetic data, ~2 minutes)

```
make demo        # starts Kafka+HDFS, generates sample, loads, trains, streams live
applications
make dashboard   # open http://localhost:5055
make test
```

Using the real Lending Club dataset

1. Create a Kaggle API token and save it as `~/.kaggle/kaggle.json`.
2. Run `scripts/download_dataset.sh` (≈375 MB) or place `accepted_2007_to_2018Q4.csv.gz` manually.
3. Run:

```
make up
make load CSV=data/lending_club/accepted_2007_to_2018Q4.csv.gz   # loans issued
before 2017 → Kafka → HDFS
make train
make live CSV=data/lending_club/accepted_2007_to_2018Q4.csv.gz   # 2017+ loans,
label stripped, scored live
make dashboard
```

The older `loan.csv` (2007–2015) layout should also load; missing columns become null (the producer prints a "columns not in file" note); pass `SPLIT=2015-01` or similar. The full file has ~2.2M rows and takes several minutes to load; use `--limit` on the producer to sample.

Make targets

Target	Action
`up`	`docker compose up -d`, then waits for HDFS WebHDFS and Kafka to be ready
`down`	`docker compose down`
`reset`	Wipes volumes, `scored.db*`, `model.joblib`, `metrics.json`, then restarts (use when switching datasets)
`sample`	Generates 60,000 synthetic rows to `data/sample_lending_club.csv`
`load`	Producer with `--issued-before $(SPLIT)` then `hdfs_sink --idle-exit 10`
`train`	`python -m lrisk.train`
`live`	Starts the scorer in the background, then streams `--issued-after $(SPLIT) --live --limit $(LIVE_N) --rate 100`, then runs the sink
`dashboard`	`python -m lrisk.dashboard.app`
`test`	`pytest -q tests`
`demo`	`up sample load train live`

Defaults: `CSV=data/sample_lending_club.csv`, `SPLIT=2017-01`, `LIVE_N=3000`.

5. Infrastructure

Service	Image / container	Ports	Notes
Kafka	`apache/kafka:3.9.0` / `lrisk-kafka`	9092	KRaft mode (broker+controller, no ZooKeeper); replication factors 1; volume `kafka-data`; healthcheck via `kafka-broker-api-versions.sh`
NameNode	`apache/hadoop:3.3.6` / `lrisk-namenode`	9870	Formats the name dir on first start only; UI and WebHDFS; volume `nn-data`
DataNode	`apache/hadoop:3.3.6` / `lrisk-datanode`	9864	Volume `dn-data`; depends on NameNode

`hadoop.env` sets `fs.defaultFS=hdfs://namenode`, replication 1, WebHDFS enabled, permissions disabled, and advertises the DataNode as `localhost` so WebHDFS redirects from the host resolve.

Kafka topics

Topic	Producer	Consumers
`loan-applications`	producer.py (key = application id)	hdfs_sink (group `hdfs-sink`), scorer (group `scorer`)

`underwriting-review`	scorer.py (flagged only)	downstream systems

Both are created with 3 partitions and replication factor 1.

6. Configuration

All settings in `config.py` are overridable through environment variables.

Variable	Default	Meaning
`KAFKA_BOOTSTRAP`	`localhost:9092`	Kafka bootstrap servers
`TOPIC_APPLICATIONS`	`loan-applications`	Incoming application topic
`TOPIC_REVIEW`	`underwriting-review`	Flagged-for-review topic
`WEBHDFS_URL`	`http://localhost:9870/webhdfs/v1`	WebHDFS endpoint
`HDFS_USER`	`root`	HDFS user name
`HDFS_RAW_DIR`	`/lending_club/raw`	Raw data directory on HDFS
`MODEL_PATH`	`models/model.joblib`	Serialized model bundle
`METRICS_PATH`	`models/metrics.json`	Training metrics
`DB_PATH`	`data/scored.db`	SQLite database
`REVIEW_THRESHOLD`	`0.5`	Flag cutoff (strictly greater than)
`PORT`	`5055`	Dashboard port

7. Modules in detail

7.1 producer.py

Streams a Lending Club CSV (plain or `.gz`) into Kafka in chunks of 50,000 rows. Options: `--live` (drop `loan_status`), `--issued-before YYYY-MM` (exclusive), `--issued-after YYYY-MM` (inclusive), `--limit`, `--rate` (msgs/s), `--chunksize`. It keeps only the columns in `RAW_COLUMNS`, reports missing ones, drops rows whose id is not numeric (the raw file ends with two summary rows), converts NaN to null, and uses lz4 compression with 50 ms linger. It retries on a full local buffer and reports delivery errors.

7.2 hdfs_sink.py and hdfs_client.py

The sink consumes `loan-applications` and writes micro-batches (default 5,000 messages or 5 s) to `/lending_club/raw/ingest_date=YYYY-MM-DD/part<partition>-<firstOffset>-<lastOffset>.jsonl`. Offsets are committed only after the write succeeds (at-least-once). File names derive from offsets so replays overwrite instead of duplicate. `--idle-exit N` stops after N idle seconds. `HDFS` is a minimal WebHDFS client (mkdirs, two-step create via 307 redirect, open, liststatus, recursive walk) built on `requests`.

7.3 features.py

Shared cleaning and feature engineering for both training and live scoring.

- **Target:** `Charged Off` / `Default` = 1, `Fully Paid` = 0; everything else (Current, Late, …) is unknown and dropped. The "Does not meet the credit policy. Status:" prefix is stripped.
- **Raw numeric:** loan_amnt, installment, annual_inc, dti, delinq_2yrs, inq_last_6mths, open_acc, pub_rec, revol_bal, revol_util, total_acc, mort_acc, pub_rec_bankruptcies, fico_range_low/high.
- **Categorical:** home_ownership, verification_status, purpose, addr_state, application_type. A fixed vocabulary from training is used at scoring time; unseen values become missing.
- **String-derived:** `term` → term_months, `emp_length` → emp_years ("< 1 year" = 0, "10+ years" = 10), `earliest_cr_line` → credit_history_years.

Engineered features

Feature	Definition
`fico`	(fico_range_low + fico_range_high) / 2
`credit_history_years`	(issue_d − earliest_cr_line) / 365.25 days
`payment_to_income`	installment × 12 / annual_inc
`loan_to_income`	loan_amnt / annual_inc
`revol_bal_to_income`	revol_bal / annual_inc
`term_months`, `emp_years`	Parsed from strings

Non-positive income yields null ratios (no division blow-up). The final model uses 25 features: 13 raw numerics (without the two FICO columns), 7 engineered, and 5 categoricals.

Excluded on purpose: post-origination columns (`total_pymnt`, `recoveries`, `last_pymnt_d`, …) because they leak the outcome, and lender-assigned `grade`, `sub_grade`, `int_rate` because they are the lender's own prior risk opinion.

7.4 train.py

- Reads all `.jsonl` under HDFS (or `--local file.csv` to skip HDFS), de-duplicates by id (keep last), keeps only loans with a known outcome.
- **Time-based split:** sorted by `issue_d`; the newest 20% (`--test-frac`) is the test set. The last 15% of the training period serves as the early-stopping validation set.
- **LightGBM params:** objective binary, learning_rate 0.05, num_leaves 63, min_data_in_leaf 100, feature_fraction 0.8, bagging_fraction 0.8 (freq 1), lambda_l2 5.0, cat_smooth 20, seed 42; up to 1,500 rounds with early stopping after 50.
- **Baseline:** a single FICO-only cutoff (lower FICO = higher risk), the status quo.
- **Outputs:** `models/model.joblib` (a `ModelBundle`) and `models/metrics.json`.

7.5 model.py

`ModelBundle` holds the booster, category vocabulary, review threshold and metadata. `score()` builds features from raw rows, returns default probabilities, and computes per-row **reason codes**: the top 3 features with positive (risk-increasing) LightGBM contributions, each with a readable label, the feature value and its impact.

7.6 scorer.py

Consumes `loan-applications` in batches (default 200 messages or 1 s), scores them, upserts into SQLite, publishes flagged events (`application_id`, `score`, `reasons`) to `underwriting-review`, then commits offsets (at-least-once; SQLite upserts make replays idempotent). Flags when `probability > threshold`. Options: `--from-beginning`, `--batch-size`, `--batch-seconds`, `--idle-exit`, `--group`.

7.7 store.py (SQLite)

Table `scores`: application_id (PK), scored_at, loan_amnt, annual_inc, dti, fico, term, purpose, score, flagged, reasons (JSON), review_status (`pending | approved | declined | more_info | not_required`), reviewed_at, reviewer_note. Index on (flagged, review_status, score DESC). WAL journal mode. Functions: `upsert_scores`, `decide` (only works for flagged rows; rejects unknown decisions), `stats` (totals, flag rate, mean score, status counts, 10-bin histogram), `list_flagged`.

7.8 Dashboard

Flask app bound to `127.0.0.1:5055`. The page shows KPI cards (applications scored, flagged, flag rate, pending, mean default probability), a score histogram with the review cutoff marked, a model-vs-FICO comparison table from `metrics.json`, and tabbed queue views (pending, approved, declined, more info, all) with approve / decline / need-info buttons and a note field. The page auto-refreshes every 15 seconds.

Route	Method	Purpose
`/`	GET	Dashboard (query `?status=`)
`/review/<application_id>`	POST	Record a decision (form fields `decision`, `note`)
`/api/stats`	GET	JSON statistics
`/api/flagged`	GET	JSON flagged list (`status`, `limit` default 200)

7.9 generate_sample.py

Produces **synthetic** data in Lending Club format (default 60,000 rows, seed 7) for demos and tests. Risk is a logistic function of FICO, DTI, term, payment-to-income, revolving utilisation, delinquencies, inquiries, public records, an explicit DTI × low-FICO interaction, credit history, purpose, home ownership and verification, plus noise. Loans from 2018 are 70% marked "Current", and realistic missingness is injected into emp_length, revol_util, mort_acc and dti.

8. Design decisions

- **No leakage:** only application-time fields are used.
- **Combined effects:** ratio features plus gradient boosting capture interactions (e.g., DTI hurts more at low FICO).
- **Time-based split:** train on older loans, test on the newest 20%, matching real deployment.
- **Explainability:** every flagged application shows its top 3 risk drivers.
- **Baseline comparison:** metrics compare against a FICO-only cutoff.
- **Delivery semantics:** at-least-once from Kafka; offset-derived HDFS file names, SQLite upserts by application id and trainer de-duplication make replays safe.

9. The 0.5 threshold

`REVIEW_THRESHOLD` (default 0.5) is the flag cutoff as specified. Defaults are a minority class (~20% of resolved Lending Club loans), so few applications score above 0.5: the review queue is small and high-precision, but **recall is low**. If underwriting capacity allows, a lower threshold (e.g., 0.3) catches more defaults. Changing it only needs a scorer restart.

10. Results on record

These come from `models/metrics.json` on disk. The row counts (45,109 ingested, 36,087 trained, 9,022 test) match the synthetic sample rather than the 2.26M-line real file, so these numbers reflect the data generator, not real credit behaviour. Re-run `make train` on the real data for meaningful figures.

0.686	0.654	0.383	0.274
ROC AUC	FICO-only AUC	PR-AUC	KS

Metric	Value
Rows ingested / trained / test	45,109 / 36,087 / 9,022
Default rate (test)	21.5%
Best iteration (early stopping)	69
Log loss / Brier score	0.482 / 0.155
At threshold 0.5: flag rate	2.7%
At threshold 0.5: precision	56.0%
At threshold 0.5: recall	7.0%
Defaults caught at the model's own review rate (2.7%): model vs FICO cutoff	7.0% vs 5.2%

Defaults caught reviewing riskiest 20%: model vs FICO cutoff	37.0% vs 33.7%

Feature importance (gain)

#	Feature	Gain	#	Feature	Gain
1	fico	14,235	14	inq_last_6mths	1,041
2	dti	6,178	15	installment	950
3	addr_state	3,094	16	delinq_2yrs	925
4	revol_util	2,852	17	total_acc	824
5	term_months	2,049	18	open_acc	616
6	credit_history_years	1,948	19	purpose	557
7	loan_to_income	1,766	20	mort_acc	531
8	payment_to_income	1,573	21	home_ownership	383
9	annual_inc	1,483	22	verification_status	348
10	revol_bal_to_income	1,431	23	pub_rec	155
11	revol_bal	1,297	24	pub_rec_bankruptcies	0
12	loan_amnt	1,226	25	application_type	0
13	emp_years	1,077			

Note that `addr_state` ranks third even though the synthetic generator assigns states at random, which suggests the model is fitting noise on that high-cardinality categorical.

11. Current state of the data on disk

Artifact	State
`data/scored.db`	3,000 scored applications; 69 flagged (66 pending, 2 more_info, 1 declined); 2,931 not_required
`data/sample_lending_club.csv`	60,000 synthetic rows
`data/accepted_2007_to_2018Q4.csv.gz`	Real Lending Club file present (~393 MB), not yet used for the model on disk (see section 10)
`models/model.joblib`, `metrics.json`	Present, trained 2026-10-13

12. Testing

`tests/test_pipeline.py` contains 6 offline tests (no Kafka or HDFS needed). All 6 passed when run for this document (`6 passed in 1.79s`).

Test	Verifies

`test_make_target_maps_outcomes_and_ignores_unresolved`	Target mapping, unresolved statuses, credit-policy prefix
`test_emp_length_parsing`	"10+ years", "< 1 year", missing
`test_build_features_formats_and_engineering`	Term/FICO parsing, "55.5%" strings, ratio and credit-history features
`test_unseen_category_becomes_missing_not_error`	Unknown category → null, no crash
`test_zero_income_does_not_explode`	Zero income → null ratios
`test_scoring_flags_only_above_threshold_and_store_roundtrip`	Scores in [0,1], flag iff score > 0.5, review status, idempotent upsert, decisions only on flagged rows, invalid decision rejected

13. Dependencies

`requirements.txt`: pandas, numpy, scikit-learn, lightgbm, pyarrow, confluent-kafka, requests, flask, joblib, pytest (unpinned). External: Docker with Compose, `make`, and optionally the `kaggle` CLI (installed by the download script).

14. Known limitations and notes

- The dashboard has **no authentication and no reviewer identity**; add both before real use. It binds to 127.0.0.1 only.
- Single-node Kafka/HDFS with replication 1: a development setup, not production.
- Recall at the 0.5 threshold is low by construction (see section 9).
- Metrics on disk come from synthetic data and are not evidence of real-world performance.
- Dependencies are unpinned and the project is not under version control.
- `.gitignore` excludes `.venv/`, `data/*.csv`, `data/*.gz`, `data/lending_club/`, `data/scored.db*`, `models/*.joblib` and `__pycache__/`.
- The Makefile's `PY` variable assumes the virtualenv at `.venv/bin/python`.